MBAi 460
Modern Software Architectures
Spring 2026

Lab Exercise #01 (v1.1)

Assignment: Accessing S3 --- web service vs. boto library
Submission: Gradescope
Policy: Individual work (no teams), AI use encouraged
Complete By: Thursday April 9th @ 11:59pm

Pre-requisites: *Lab 00; lecture recording for "AWS S3 as an example".*

Overview

S3 is Amazon's ***Simple Storage Service***, used for low-cost cloud storage of files. These are usually large files: documents, images, and videos. S3 is organized into storage *buckets*, where every file within a bucket has a unique *key*. S3 is not a file system --- you cannot open and read the contents of a file. Instead, it's a storage service where you can upload and download files. This is why the contents of an S3 bucket are generally referred to as objects, not files.

S3 is implemented as a web service. In this lab exercise you're going to interact with S3 in two ways: (1) directly as a ***web service***, and (2) indirectly via Amazon's ***boto3*** Python library. In a real application you would normally use the boto3 library since it's easier and less error prone. We want you to access the web service directly so you understand how the boto3 library (and S3) is implemented.

Setting up your programming environment

We are recommending the use of Docker as your base programming environment; Lab 00 provides the setup instructions for both Docker and VS Code. However, this lab exercise requires only ***Python3*** and the installation of two libraries (***boto3*** and ***requests***), so you can work without Docker if you'd prefer to configure your own Python environment. The instructions here assume you are working within the provided Docker environment; if not, simply read through the instructions and you'll see notes on what to do.

If you haven't already, review the lecture recording for "AWS S3 as an example"; you can find the link to the recording (and the underlying PPT and demo code) on Canvas under "Lecture Recordings". The lecture presents two demos, accessing S3 using the *boto3* library, and then as a *web service*. The demos in the recording are based on 3 program files:

1. main-boto.py

2. main-s3.py
3. s3-config.ini

These are working Python examples that access one of the buckets in our staff AWS account (information about which can be found in “s3-config.ini”). These files are provided for you in the docker container; if you are not using docker, you can download the files from dropbox.

Locate the "mbai460-client-main" folder representing your docker container. Open a Terminal / PowerShell window in this folder, and run the docker container. Then change (cd) into the **labs** folder, and then into the **lab01** sub-folder. Use the ls command to list the contents of the folder, and you'll see the 3 files:

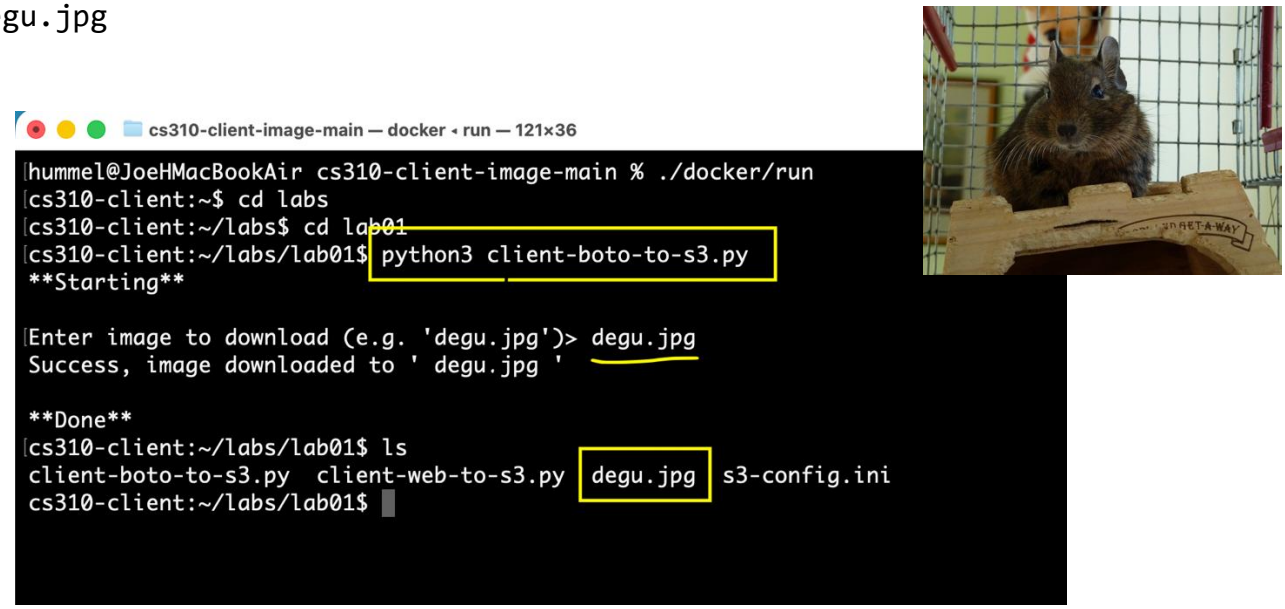
```
hummel> ./docker/run
client:~$ cd labs
client:~/labs$ cd lab01
client:~/labs/lab01$ ls
main-boto.py  main-s3.py  s3-config.ini
client:~/labs/lab01$
```

The files are badly named, so the first step is to rename the Python files (this is a Gradescope requirement). In the command window use the move command "mv" to rename the files:

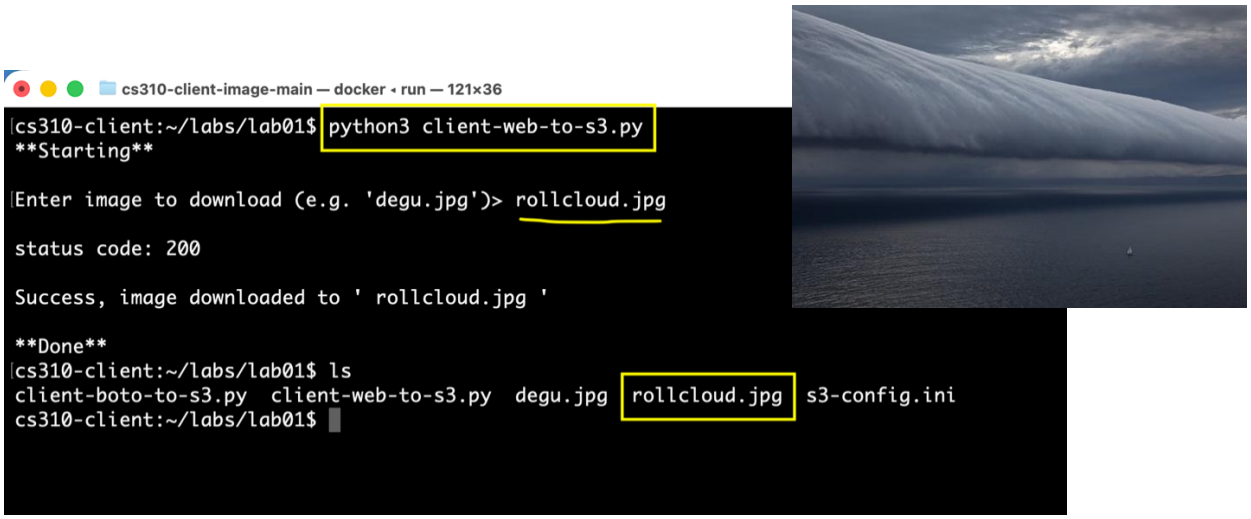
```
1. mv main-boto.py client-boto-to-s3.py
2. mv main-s3.py client-web-to-s3.py
```

The provided files should work as given, so this would be a good time to test things. Let's run the boto3 version and download an image of a degue (small rodent):

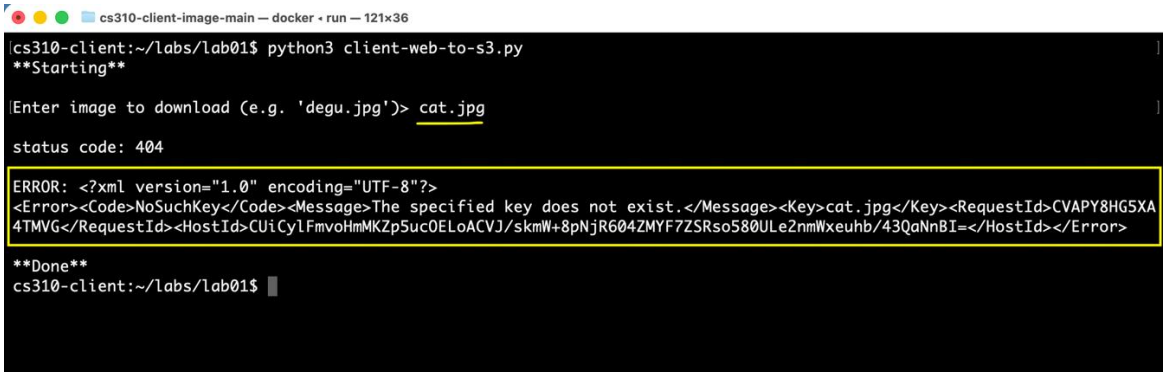
```
3. python3 client-boto-to-s3.py
4. degu.jpg
```



Navigate to the lab01 folder on your local PC, double-click on the file “degu.jpg”, and you should see a picture of Piper ☺. Now let’s test the web service version of the program:



Finally, run either version and see what happens when you enter the name of a file that doesn't exist in our S3 bucket (e.g. "cat.jpg"):



Assignment

Some of the files in our S3 bucket do not have file extensions. When you download these files to your local computer, it will be unable to open them due to the missing file extension. You can test this yourself: run either version and download the file "A" from our S3 bucket. It will download successfully, but you won't be able to open file "A" in a meaningful way.

The solution is to look at the ***content type*** of the download, which tells you the type of data that was downloaded. This might be "application/pdf", or perhaps "image/jpeg". Based on the content type, we will add the appropriate extension to the filename and be able to view the file locally.

Let's start with the version that calls the S3 web service directly…

Calling the S3 web service directly

We're going to edit and make some changes to the provided program files; while any editor will work, we recommend ***VS Code*** *(you were asked to install in Lab 00)*. Open and review the source code for "client-web-to-s3.py". There are 6 major steps in the program code:

1. Opens "s3-config.ini" and retrieves the URL for the web service
2. Prompts the user for a filename
3. Makes a "get" request to download the file. To call a web service you build a URL that contains the web server address followed by / followed by the data you want or the operation to perform. Example: to download "degu.jpg" from our S3 bucket, the actual URL is:

   https://nu-cs-msa-s3-photoapp.s3.us-east-2.amazonaws.com/degu.jpg

   Confirm this by adding *print(url)* to the program. Run, test, then comment out the *print( )* call.

4. Checks the status code, which is the integer code returned by the web server. 200 denotes success, anything else denotes an error, e.g. 404 means not found.
5. If a status code of 200 is returned, we write the ***contents*** of the response to a binary file. The contents are the image in its binary form.
6. If some other status code is returned, we print the ***text*** of the response, which is a string.

Make the following changes to the program. First, assume the user is going to enter a filename without an extension. You can test with the filenames A, B, C, and D --- these will all download successfully. Upon a successful download, retrieve the ***Content-Type*** from ***response.headers***, and append the appropriate file extension to the local filename based on the following supported content types:

image/jpeg => .jpg

text/plain => .txt

application/json => .json
application/pdf => .pdf
application/xml => .xml
application/zip => .zip

text/x-python => .py
text/x-python-script => .py
application/x-python-code => .py

If the content type is missing, or is some other type, append ".unknown". Run and test with A, B, C, and D, which should have the extensions shown here ---------->

You should be able to open each file successfully after downloading. [**Note**: you need to match this output exactly; notice the prompt has changed, and the print( ) of the filename is '…' with no spaces. Python's ***f"…"*** strings are very helpful for formatted output.]

Second, modify the program to output a better error message. As given, upon an error the program output is an XML string, which contains the error message but is generally hard to read. Instead, parse the returned XML in response.text, extract the error message, and then output the URL and the error message as follows:

```
cs310-client-image-main — docker ◂ run — 121×43
cs310-client:~/labs/lab01$ python3 client-web-to-s3.py
**Starting**

Enter image to download without extension> cat

status code: 404

ERROR:
 URL: https://nu-cs-msa-s3-photoapp.s3.us-east-2.amazonaws.com/cat
 Msg: The specified key does not exist.

**Done**
```

Feel free to use AI to help you figure out how to parse the XML. A common approach is to

```
import xml.etree.ElementTree as ET
```

And then use *xml_doc = ET.fromstring( )* to parse the XML, and then *xml_doc.find('Message').text* to extract the text of the error message.

Finally, given the unpredictable nature of the internet / WiFi / networking / computers, we're going to implement a ***retry*** protocol for calling the web service. The typical approach when building networking applications is to retry on a failure, at most 3 times, with a random-like wait before trying again. This implies we are going to replace the single call

```
response = requests.get(url)
```

With a retry loop:

```
retry = 1
max_retries = 3

while retry <= max_retries:
  try:
    response = requests.get(url)
    .
    .
  except Exception as e:
    .
    .
  finally:
    .
    .
```

If the response returns a status code of 200 or 404, then the call was a success and you should break out of the retry loop. Otherwise you sleep *retry* seconds, increment retry, and try again. If an exception occurs, you check to see if you have tried 3 times. If not, you sleep retry seconds, increment retry, and try again. [**Hint**: use the *finally* block since the sleep and increment are common in both try and except.]

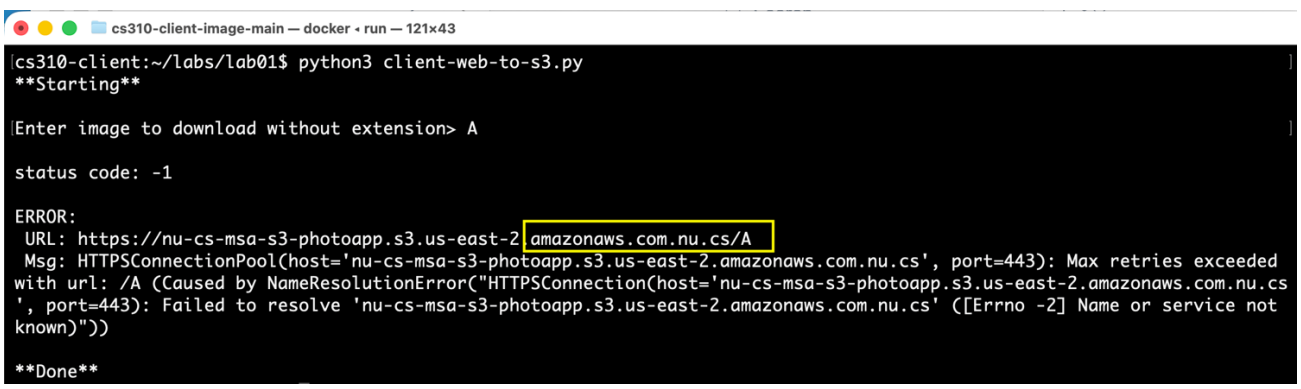
If you get an exception and retry == max_retries, then the goal is to build a fake response object with a status code of -1 and an XML string containing the exception information. This allows for the program to continue execution, eventually outputting the status code and error message as before. Feel free to use AI to help you, but here's how we did it (***e*** is the Exception object):

```
response = requests.models.Response()
response.status_code = -1

root = ET.Element("Error")
ET.SubElement(root, "Message").text = str(e)
response._content = ET.tostring(root, encoding='UTF-8')
```

[**NOTE**: do not take the simpler approach and merely exit the program in this case. The goal is a more general approach we can reuse in future assignments. We will not grade solutions that exit abnormally.]

To test your retry strategy, intentionally introduce an error… You can disable your internet, forcing a timeout. Or you can alter the web server name, which is what we did here (see the URL in the error message, where we appended ".nu.cs" to the endpoint):



Make sure your output matches exactly for Gradescope purposes. Good job!

Calling S3 indirectly via the boto library

Open and review the source code for "client-boto-to-s3.py". The code contains a similar set of steps for interacting with S3:

1. Opens "s3-config.ini" and retrieves the bucket information for configuring boto. Creates a ***bucket*** object that represents the staff S3 bucket, and to use for calling boto functions.
2. Prompts the user for a filename.
3. Calls S3 via the ***download_file( )*** function to download the image. Internally, this does what we did

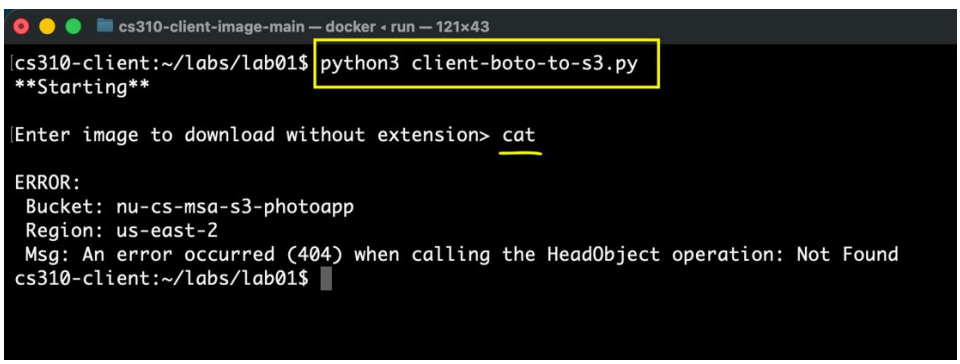
earlier --- makes a get request to the underlying web service.

4. If an error occurs, this implies the download failed. The error is caught and an error message is output.

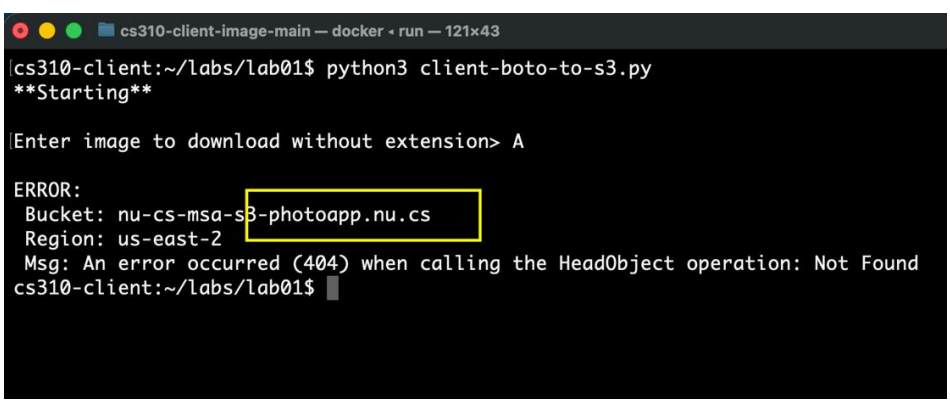
As you can tell, the code is simpler than the explicit call to S3 using the web service. In fact, let's configure the ***retry*** protocol for boto to use a maximum of 3 attempts. Update the provided code to configure boto as follows:

```
s3 = boto3.resource(
        's3',
        region_name=region_name,
        config = Config(
            retries = {
                'max_attempts': 3,
                'mode': 'standard'
            },
            signature_version=UNSIGNED # enable access to public objects
        )
    )
```

That's update #1. As for update #2, the error message returned by boto is already a printable string. However, let's change the format to match what we did for the web service version, outputting the bucket and region names:



Notice the prompt is updated as well. Test what happens when the internet is disconnected, or the wrong bucket name is used:



In this case the bucket was not found (though it doesn't say that explicitly). Finally, update #3 is to deal with

downloads that have no file extensions. As before, assume the user will enter a filename without an extension. To obtain the content type, use the ***s3*** object to request the object's meta-data:

```
object_metadata = s3.Object(bucket_name, imagename)
```

The content_type is a field of the metadata object. You'll need to do this before you download the file, so that you can build the correct local filename. Here are the results for existing downloads A, B, C, and D:

```
cs310-client-image-main — docker ◂ run — 121×43
cs310-client:~/labs/lab01$ python3 client-boto-to-s3.py
**Starting**

Enter image to download without extension> A

Success, image downloaded to 'A.jpg'

**Done**
cs310-client:~/labs/lab01$ python3 client-boto-to-s3.py
**Starting**

Enter image to download without extension> B

Success, image downloaded to 'B.txt'

**Done**
cs310-client:~/labs/lab01$ python3 client-boto-to-s3.py
**Starting**

Enter image to download without extension> C

Success, image downloaded to 'C.py'

**Done**
cs310-client:~/labs/lab01$ python3 client-boto-to-s3.py
**Starting**

Enter image to download without extension> D

Success, image downloaded to 'D.pdf'

**Done**
cs310-client:~/labs/lab01$
```

That's it!

Grading and Electronic Submission

Gradescope will be our primary mechanism for collecting and grading lab and project assignments. Lab code does not need to be commented unless otherwise specified, although it's always a good idea to put your name in the header comment at the top of the file; AI use is encouraged to help you complete the assignment. Individual work only, no teams.

If you are working in Docker, you can submit your Python files to gradescope as follows:

```
/gradescope/gs submit 1288073 7906072 *.py
```

If the above fails, then gradescope is not yet accepting submissions; come back and try again later (Gradescope generally opens a few days before the due date). If you are working outside Docker, you can submit from your LOCAL computer by viewing your folder locally using Mac/Windows/Linux, open Gradescope (via Canvas or https://gradescope.com), open "Lab 01 – writing clients", and then drag-dropping your Python files.

The goal is an autograder score of 100/100; be sure to check the results of your submission, do not simply submit and walk away, assuming all is well. If you get a score < 100, look at the Gradescope output, fix your errors, and resubmit. You have unlimited Gradescope submissions in this assignment.